AES Specification

1. 功能介绍

该模块完成 AES 的数据加密(encipher)/解密(decipher)功能。软件配置好信息后(KEY，源数据地址（明文/密文），目的数据地址（密文/明文）)，它直接从源数据地址空间读取源数据块，AES 加密/解密后写入目的数据地址空间。软件配置寄存器采用 APB slave 接口，数据传输采用 AXI master 接口。

具体功能如下：

- 支持 AES 加密/解密算法；支持 ECB，CBC 模式；
- KEY 长度支持 128bit，256bit；
- 13(128bit key)/17(256bit key)个周期完成一个 128bit 数据块的加密/解密；
- 内嵌 DMA 模块，主动进行数据读写，最大 burst length 4；

为了提高 AXI bus 上的数据读写带宽，使用了如下 AXI 总线的特性：

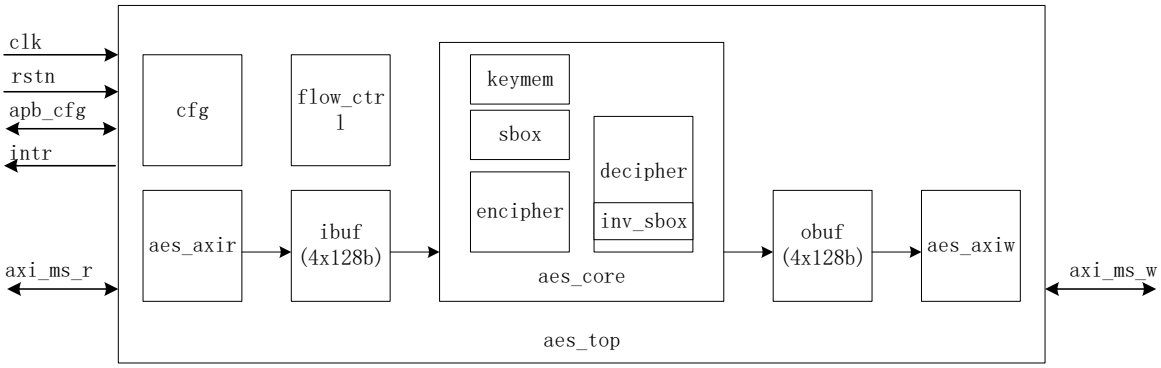
- Read/Write channel 并发操作；
- Read command outstanding；

本设计综合考虑了 AXI 总线 data 传输时的总线等待(wait cycle)与处理效率的关系，即 AXI 总线数据传输率与 AES 加解密数据吞吐率的平衡，折衷选择了适合的内部 buffer 深度（目前是 4x128bit，缓存 4 个 AES 数据块）。

注：

1) 如果 AXI read latency 比较大，可以适当增加 input buffer 的深度。
2) 假设 AXI write data latency 很小，没有做 write command outstanding。如果实际 axi bus 系统的 write data latency 比较大，需要修改代码实现 write command outstanding，必要时可是适当增加 output buffer 的深度。

设计框图如下图所示：



2. IO 功能定义

Name	Direction	Bits	Description
clk	I	1	时钟输入。
rstn	I	1	异步复位输入，低有效。
intr	O	1	中断输出，high level active。
axi_ms_inf	I/O	NA	标准的 AXI master 接口(32bits, maximal burst length is 4)。
apb_cfg	I/O	NA	标准的 APB salve 接口。

AXI 接口时序：见 AMBA 标准协议：《AMBA™ AXI Protocal》，v1.0;
APB 接口时序：见 AMBA 标准协议：《AMBA™ 3 APB Protocal》，v1.0;

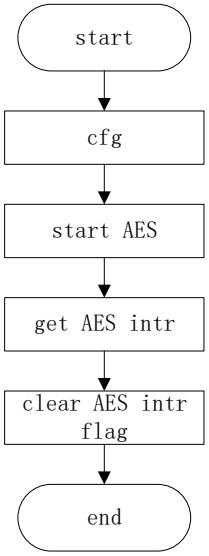
注：AMBA 协议是 ARM 公司定义的总线协议。

3. 寄存器定义

addr	bit	name	access	description
00	0	aes_sof	r/w	SW 写 1 启动一个数据块的 AES 处理。SW 读返回 0。
	3:1	NA	r	Reserved。
	4	keg_gen	r/w	SW 写 1 启动一次 AES 中 key expand 过程。SW 读返回 0。
	31:5	NA	r	Reserved。
04	0	enc_dec	r/w	1: encipher mode; 0: decipher mode。
	3:1	NA	r	Reserved。
	5:4	blk_mode	r/w	0: ECB; 1: CBC; 2: CFB; 3: OFB。CFB/OFB 模式暂不支持。
	7:6	NA	r	Reserved。
	9:8	key_len	r/w	0: 128bit key; 1: 196bit key; 2: 256bit key。
	31:10	NA	r	Reserved。
8	31:0	NA	r	Reserved。
0C	0	intr	r/w	1: AES 结束一个数据块处理。SW 写 0 清楚中断。
	31:1	NA	r	Reserved。
10	31:0	blk_base	r/w	AES 读取的源数据的系统地址，byte 单位。必须是 128bit 对齐的地址，即：blk_base[3:0] =

				4'b0000。
14	31:0	code_base	r/w	AES 处理后写出数据的系统地址，byte 单位。必须是 128bit 对齐的地址，即：code_base[3:0] = 4'b0000。
18	31:0	blk_len	r/w	AES 处理的数据块的大小，byte 单位，从 0 计数。必须是 16byte 的整数倍，即：blk_len[3:0] = 4'b1111。
1C	31:0	NA	r	Reserved。
20	31:0	iv[31:0]	r/w	initial vector。
24	31:0	iv[63:32]	r/w	initial vector。
28	31:0	iv[95:64]	r/w	initial vector。
2C	31:0	iv[127:96]	r/w	initial vector。
30	31:0	NA	r	Reserved。
34	31:0	NA	r	Reserved。
38	31:0	NA	r	Reserved。
3C	31:0	NA	r	Reserved。
40	31:0	key[31:0]	r/w	key vector。
44	31:0	key[63:32]	r/w	key vector。
48	31:0	key[95:64]	r/w	key vector。
4C	31:0	key[127:96]	r/w	key vector。
50	31:0	key[159:128]	r/w	key vector，valid for 256bit key length。
54	31:0	key[191:160]	r/w	key vector，valid for 256bit key length。
58	31:0	key[223:192]	r/w	key vector，valid for 256bit key length。
5C	31:0	key[255:224]	r/w	key vector，valid for 256bit key length。

4. HW Control Flow



5. 主要子模块功能

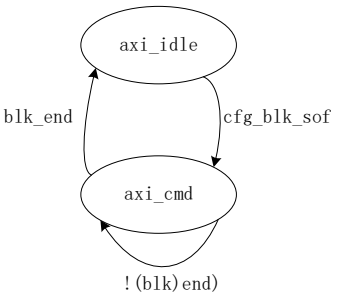
5.1. aes_cfg

该模块含所有配置寄存器。该模块实现 apb slave 接口，通过 apb 接口完成配置寄存器的读写操作。其中 pready 固定为 1'b1，读写均不会插入 wait cycle。对外的处理中断生成与清除也在这个模块完成。

5.2. aes_axir

该模块通过 axi read 接口从系统内存读取 AES 处理的源数据（加密模式是明文，解密模式是密文）并写入 ibuf，固定发 4 beat x 32bit 的 read burst。需要 ibuf 有剩余空间时，才发出 arvalid，保证 read 数据能立即被接收，即 rready==1'b1。目前 ibuf 容量为 4x128bit，最多 outstanding 4 个 read command。

模块内含一个简单状态机，控制 axi read cmd 的发送。为了实现 read command oustanding，此状态机仅以 axi read cmd channel 的 handshake 来控制状态机的走向，跟 axi read data channel 的 handshake 无关。

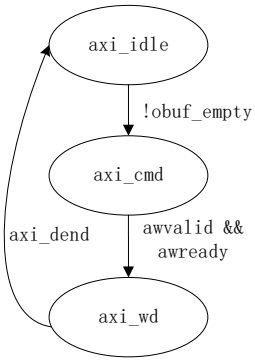


axi_idle：等待 cfg_blk_sof 的 1T high pulse，进入 axi_cmd 状态开始一包数据的处理；
axi_cmd：每当 input buffer 不满时（满判断跟 read cmd 的发送对齐，不跟 read back data 对齐）就把 arvalid 置 1，开始一次 axi read cmd channel 的握手；当所有源数据都处理完后，进入 axi_idle 状态；

5.3. aes_axiw

该模块从 obuf 读取数据，并通过 axi read 接口往系统内存写 AES 处理后的数据（加密模式是密文，解密模式是明文），固定发 4 beat x 32bit 的 write burst。。需要 obuf 非空时，才发出 awvalid；在 write cmd channel 的 hansshake 完成后立即开始 write data channel 的 handshake，在 write data channel 传输 4 beat 32bit 数据的时间内，wvalid 一直是 1'b1。

模块内含一个简单状态机，控制 axi write cmd 与 write data channel 的 handshake 过程。目前 2 者是在一个状态机中串行执行的，没有利用 write cmd oustanding 的特性。如果实际 axi bus 系统的 write data latency 比较大，需要修改代码实现 write command outstanding。



axi_idle：等待(!obuf_empty)，进入 axi_cmd 状态；
axi_cmd：发送 awvalid=1'b1，等到 write cmd channel 的 handshake 结束，进入 axi_wd 状态；
axi_wd：wvalid=1'b1，完成 4 beat x 32bit 数据写的 handsake，进入 axi_idle 状态；

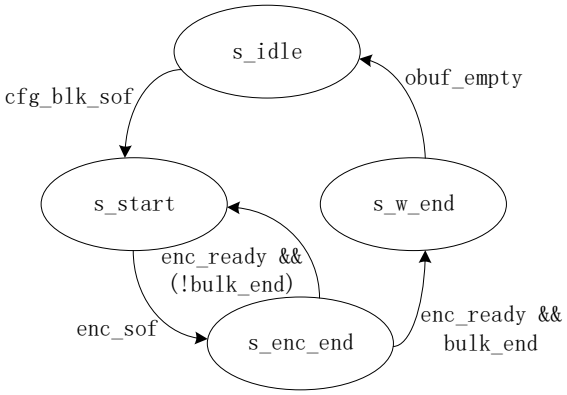
注：此处无需最终数据块的总大小，因为：每 128bit 数据通过 axi read 进来做加解密处理后，都会有 128bit 结果数据写入 obuf。obuf 里面的数据肯定是需要全部依次写入系统内存的。

5.4. aes_flow_ctrl

该模块完成以下功能：

1) 监测一个数据包加解密的启动与完成。一个数据包加解密完成后，set_blk_end 发送 1T high pulse 给 aes_cfg 模块；
2) 从 ibuf 读取数据，送给 aes_core 做加解密处理，并 enc_sof 发送 1T high pulse 启动一个 128bit 数据的加解密；ibuf 的 ibuf_rd 信号故意推迟到该 block 加解密结束后再拉高 1T，保证模块输出信号 enc_text[127:0]在整个 block 处理过程中保持不变，方便 CBF/OFB 模式的计算；
3) 监测 aes_core 的 block end 信号，在 enc_ready=1 时把 aes_core 输出的结果数据：enc_code[127:0]写入 obuf；
4) 生成 first_blk/blk_end 等状态与控制信号；

模块内含一个简单状态机，状态定义与跳转关系如下：



s_idle：等待 cfg_blk_sof 的 1T high pulse，进入 s_start 状态开始一包数据的梳理；
s_start：等待条件" key_ready && (!obuf_full) && (!ibuf_empty)"成立，启动一个 128bit 数据块的加解密处理；
s_enc_end：等待 aes_core 处理完成一个 128bit block 后发送 enc_ready=1；如果还有 128bit block 需要处理则进入 s_start 状态；如果没有 block 需要处理则进入 s_w_end 状态；
s_w_end：等待 aes_axiw 模块把 obuf 里面的数据全部通过 axi 接口写出；则整个 bulk 数据包的处理就完成了；

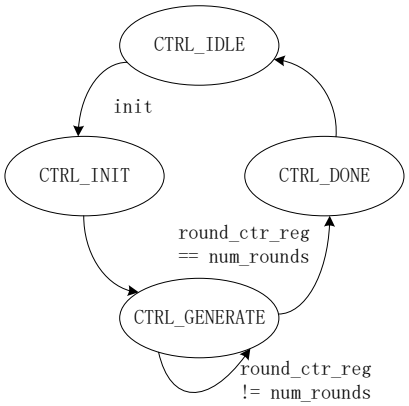
5.5. aes_key_mem

该模块使用输入的 128bit/256bit 初始 KEY 做 key expansion 生成加解密过程中需要的总计 11/15 个 128bit 的 round KEY，并保存在该模块中。每个周期生成 1 个 128bit 的 round KEY。加解密过程中，根据输入" round"的计数值 MUX 出该轮需要的 round KEY。

在 128bit 的初始密钥长度下，初始密钥在输入"key[255:128]"中；上一个 128bit group 的轮密钥保存在寄存器"prev_key1_reg[127:0]"中。每个周期新生成的 128bit round KEY 中的每个 32bit 都要用到" prev_key1_reg[127:0]"中跟自己是 4 个 32bit 距离的那个 32bit 数据。

在 256bit 的初始密钥长度下，初始密钥在输入"key[255:0]"中；上一个 256bit group 的轮密钥保存在寄存器"{prev_key0_reg[127:0], prev_key1_reg[127:0]}"中。每个周期新生成的 128bit round KEY 中的每个 32bit 都要用到"{prev_key0_reg[127:0], prev_key1_reg[127:0]}"中跟自己是 8 个 32bit 距离的那个 32bit 数据。

模块内含一个简单状态机，状态定义与跳转关系如下：



CTRL_IDLE：空闲状态。等待 init=1'b1，跳转到 CTRL_INIT；把 rcon_reg 置 0x8D；
CTRL_INIT：只呆 1 个周期，把 round_ctr_reg 置 0x00，立即跳转到 CTRL_GENERATE；
CTRL_GENERATE：在该状态呆 11/14 个周期，每个周期生成一个 128bit 的 round key；在此

状态 round_ctr_reg 每个周期累加 1；在此状态，为 128bit KEY 模式时，rcon_reg 每 1 个周期更新一次；为 256bit KEY 模式时，rcon_reg 每 2 个周期更新一次；
CTRL_DONE：只呆 1 个周期，立即跳转到 CTRL_IDLE。

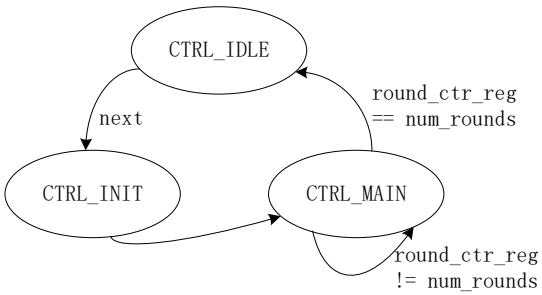
5.6. aes_encipher_block

该模块的一次启动完成 1 个 128bit 数据块的加密。在 128bit/256bit 密钥模式下，分别需要循环 11/15 次，并使用 11/15 个 round KEY 来完成 128bit 数据块的加密，每个周期完成一次循环。加上状态机进入与退出的 cycle 消耗，在 128bit/256bit 密钥模式下每 13/17 个周期完成 1 个 128bit 数据块的加密处理。该模块用组合逻辑实现：GF 乘法，ShiftRow，MixColumns，与 round KEY 的 XOR 操作；组合逻辑从“aes_key_mem”模块查表完成 SubByte 操作。

关键信号如下：

1) ”round_ctr_reg”：循环计数器，从 0 计数到 10/14。
2) “block_w0_reg”/” block_w1_reg”/” block_w2_reg”/” block_w3_reg”：每轮循环更新的 128bit 状态；
3) “enc_ctrl_reg”：简单控制流程的 FSM；

“enc_ctrl_reg”的状态机定义与跳转过程如下：



CTRL_IDLE：空闲状态，等待 next=1’b1 就跳转到 CTRL_INIT 状态，同时把 round_ctr_reg 清 0；
CTRL_INIT：只呆 1 个周期，使用第 1 个 round KEY 做输入与 round KEY 的 XOR 操作；round_ctr_reg 在该状态加 1。
CTRL_MAIN：呆 10/14 个周期，使用第 2~11/15 个 round KEY 做计算。round_ctr_reg 在该状态的每个周期都加 1。

5.7. aes_decipher_block_fast

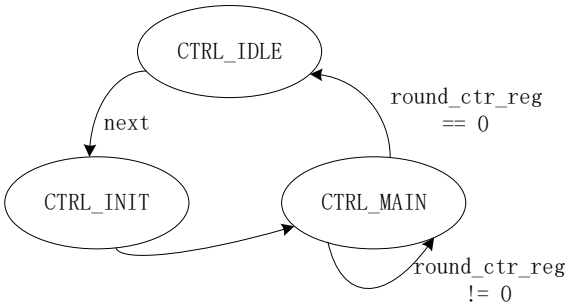
该模块的一次启动完成 1 个 128bit 数据块的解密。在 128bit/256bit 密钥模式下，分别需要循环 11/15 次，并使用 11/15 个 round KEY 来完成 128bit 数据块的加密（解密过程，需要逆序使用 round KEY，即最先使用第 11/15 个 round KEY，最后使用第 1 个 round KEY），每个周期完成一次循环。加上状态机进入与退出的 cycle 消耗，在 128bit/256bit 密钥模式下每 13/17 个周期完成 1 个 128bit 数据块的解密处理。该模块用组合逻辑实现：GF 乘法，InvShiftRow，InvMixColumns，与 round KEY 的 XOR 操作；通过组合逻辑查表完成 InvSubByte 操作。

关键信号如下：

1) ”round_ctr_reg”：循环计数器，从 11/15 递减到 0；

2) “new_block[127:0]”：每轮循环更新的 128bit 状态；
3) “dec_ctrl_reg”：简单控制流程的 FSM；

“dec_ctrl_reg”的状态机定义与跳转过程如下：



CTRL_IDLE：空闲状态，等待 next=1’b1 就跳转到 CTRL_INIT 状态，同时把 round_ctr_reg 置 10/14；
CTRL_INIT：只呆 1 个周期，使用第 11/15 个 round KEY 做输入与 round KEY 的 XOR 操作；round_ctr_reg 在该状态减 1。
CTRL_MAIN：呆 10/14 个周期，使用第 10/14~1 个 round KEY 做计算。round_ctr_reg 在该状态的每个周期都减 1。

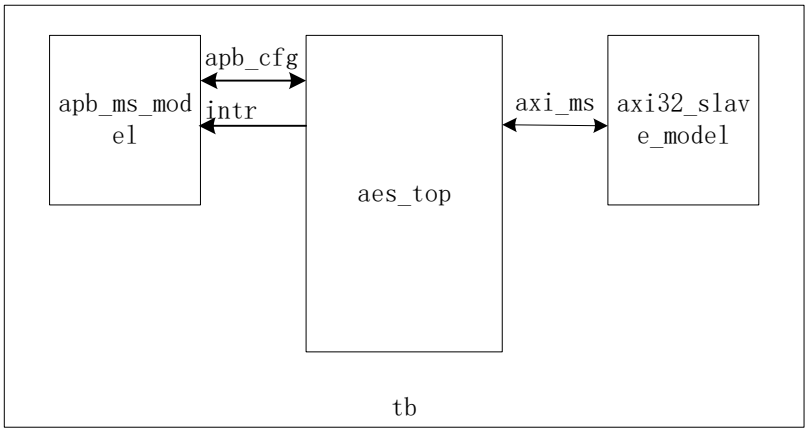
5.8. aes_core

该模块完成如下功能：

1) 调用子模块：aes_encipher_block, aes_decipher_block_fast, aes_key_mem, aes_sbox_lut；
2) 生成 init_state 信号，表示正在做 key expansion。当 init_state=1’b1 时，不能做加解密运算；用 init_state 信号选择对 encipher 还是 key expansion 模块做 SubByte 操作；
3) 对 encipher 的输入做 XOR 与 MUX 操作；
4) 对 decipher 的输出做 XOR 与 MUX 操作；
5) MUX encipher/的 loop counter，去 aes_key_mem 读取对应的 round key；
6) MUX encipher/decipher 的 128bit 输出结果与 block 结束状态；

6. RTL Simulation

simulation 基本思路：用 c model 产生测试输入数据与对比结果，并保存在二进制文件中（见 sim_case 目录）。RTL 仿真时，apb_ms_model 读取该文件，获取 AES 加解密信息并通过 apb 接口配置进 DUT。在一整块数据加解密结束后，用 RTL 输出与 C model 输出进行对比检查。



C model 输出的二进制对比文件的数据格式：

对比文件格式		
	内容	大小
header	enc_dec	1B
	blk_mode	1B
	key_len	1B
	test_bulk_num	4B
	KEY	16B or 32B
	IV	16B
数据块 1	blk_len	4B
	aes_in_data	blk_len B
	aes_out_data	blk_len B
数据块 2	blk_len	4B
	aes_in_data	blk_len B
	aes_out_data	blk_len B
●●●		
数据块 N	blk_len	4B
	aes_in_data	blk_len B
	aes_out_data	blk_len B

7. Improvement

- 减少加密一个 128bit block 需要的周期数（到 12 或 11 个周期）；
- RTL sim/C model 增加更多计算中间结果打印，方便 debug；
- 跟 SHA-256 IP 配合（可能需要做接口的修改），做成一个带数据完整性校验的安全数据收发系统（具体见视频：63.03sha256 HW Spec.与架构讲解，22 分 20 秒。）；
- 编码增加 CFB, OFB 模式的支持；
- 增加 196bit key length 的支持；
- 增加解码功能的支持；
- EBC 模式，考虑高吞吐率 HW 结构；
- 考虑降低数据吞吐率，同步较少 HW area 的设计架构；
- 改 AHB Master 接口；
- 研究加解密的攻击（比如旁路攻击，注入攻击）与抗攻击的原理与 HW 实现；

8. 参考资料

1) AES 英文协议：《ADVANCED ENCRYPTION STANDARD》。
2) aes_core 部分的代码，参考 github：https://github.com/secworks/aes/tree/master，做了修改提升了单周期运算能力。